



Șirul lui Fibonacci | Algoritmi pe șiruri

Elementul majoritar · Subsecvența de sumă maximă

Andrei Mihai Ciobanu

Algolymp Summer School

Șirul lui Fibonacci

Elementul majoritar

Subsecvența de sumă maximă

Recapitulare

Resurse

Probleme de antrenament

Șirul lui Fibonacci

Șirul lui Fibonacci

$$F[0] = 0, \quad F[1] = 1, \quad F[i] = F[i-1] + F[i-2] \text{ pentru } i \geq 2$$

i	0	1	2	3	4	5	6	7	8
$F[i]$	0	1	1	2	3	5	8	13	21

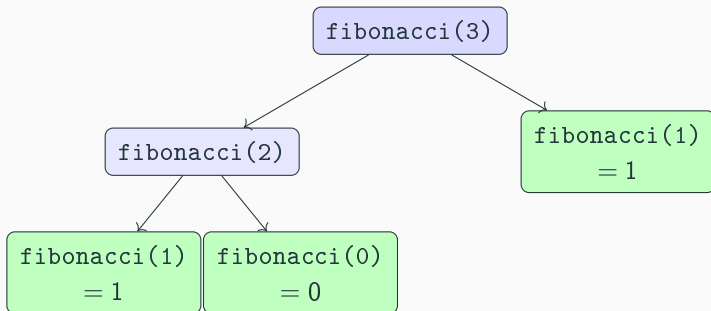
Fiecare termen e suma celor doi dinaintea lui. Întrebarea cheie: ca să calculăm $F[i]$, de ce informație avem **cu adevărat** nevoie?

Varianta recursivă naivă

```
1 long long fibonacci(int n) {  
2     if (n < 2) {  
3         return n;  
4     }  
5     return fibonacci(n - 1) + fibonacci(n - 2);  
6 }
```

- Codul arată exact ca formula, dar e **foarte lent**.
- Problema: aceleași subprobleme se **recalculează de mai multe ori** — `fibonacci(2)` e recalculat ori de câte ori e nevoie de el.
- `fibonacci(40)` face ≈ 330 de milioane de apeluri; `fibonacci(100)` nu se termină în timp rezonabil.

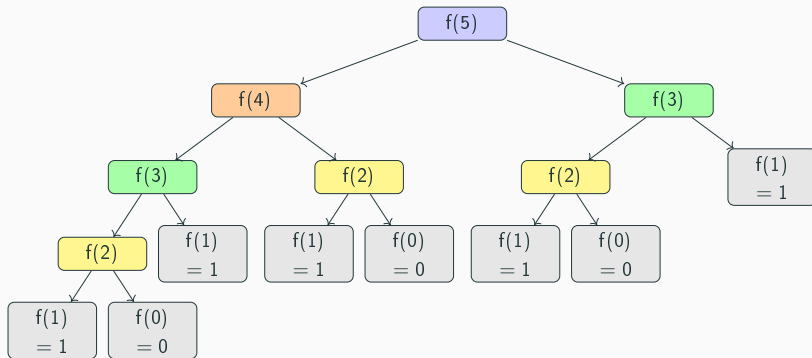
Arborele de apeluri — fibonacci(3)



Fiecare nod non-terminal generează exact 2 apeluri recursive.

Total: **5 apeluri** pentru $n = 3$.

Arborele de apeluri — fibonacci(5)



Total: **15 apeluri**. Noduri de aceeași culoare = calcule repetate.

Câte apeluri face fibonacci(n)?

n	1	2	3	4	5	6	7	8	9	10
$F[n]$	1	1	2	3	5	8	13	21	34	55
Apeluri	1	3	5	9	15	25	41	67	109	177

Observăm că numărul de apeluri pentru n este exact $2 \cdot F[n+1] - 1$.

Numărul de apeluri crește **mult mai rapid** decât valorile Fibonacci
→ creștere exponențială.

Creștere exponențială și golden ratio (φ)

Dacă împărțim numărul de apeluri de la x la cel de la $x - 1$:

x	$\text{apeluri}(x) / \text{apeluri}(x - 1)$
3	$5 / 3 \approx 1,67$
5	$15 / 9 \approx 1,67$
7	$41 / 25 \approx 1,64$
9	$109 / 67 \approx 1,63$
10	$177 / 109 \approx 1,62$

Raportul se apropie de $\varphi \approx 1,618\dots$ — **golden ratio**. Cu cât x e mai mare, cu atât raportul converge mai precis.

Fun fact: Numărul de aur și formula lui Binet

Numărul de aur

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1,618 \dots$$

Formula lui Binet

$$F[n] = \frac{\varphi^n - \psi^n}{\sqrt{5}}, \quad \text{unde } \psi = \frac{1 - \sqrt{5}}{2} \approx -0,618$$

- Deoarece $|\psi| < 1$, pentru n mare: $F[n] \approx \varphi^n / \sqrt{5}$
- Asta explică de ce varianta recursivă naivă e $O(\varphi^n)$ — adică exponențială
- φ apare și în natură: frunze, petale, spiralele scoicilor

Varianta iterativă

Ca să calculăm $F[i]$, ne trebuie doar **ultimele două** valori: $F[i - 2]$ și $F[i - 1]$.

```
1  const long long MOD = 1000000007;
2
3  long long fibonacci(int t) {
4      long long f0 = 0, f1 = 1;
5      for (int i = 0; i < t; i++) {
6          long long next = (f0 + f1) % MOD;
7          f0 = f1;
8          f1 = next;
9      }
10     return f0;
11 }
```

După i pași, $f0$ memorează $F[i]$ — de aceea, după t pași, $f0$ este exact $F[t]$.

Variantă	Timp	Spațiu
Recursivă naivă	$O(\varphi^n)$	$O(n)$ (stivă)
Iterativă	$O(t)$	$O(1)$

Bonus

Dacă t e extrem de mare (ex. 10^{18}), există metode $O(\log t)$ prin **exponențiere matriceală** sau *fast doubling*. Nu sunt necesare acum, dar ideea de înjumătățire va reapărea.

Elementul majoritar

Ce este un element majoritar?

Element majoritar strict

O valoare care apare de **mai mult de $n/2$** ori într-un şir cu n elemente.

Exemplu: $v = [7, 2, 7, 7, 4]$ — valoarea 7 apare de 3 ori din 5, deci este majoritar ($3 > 5/2$).

Contraexemplu: $v = [1, 1, 2, 2]$ — cele mai frecvente valori apar de 2 ori, dar 2 nu e mai mare decât $4/2$, deci **nu există** element majoritar.

Atenție

“Cea mai frecventă valoare” **nu** este același lucru cu “element majoritar strict”.

Povestea voturilor

Imaginați-vă o alegere: n votanți, fiecare votează un candidat. Un candidat câștigă dacă primește **mai mult de jumătate** din voturi — adică are **majoritate absolută**.

Regula de anulare

Oricare doi votanți care au votat candidați **diferiți** se anulează reciproc și ies amândoi din joc.

- Dacă există un candidat majoritar, are prea mulți susținători pentru a fi anulat complet — vor rămâne mereu unii care l-au votat
- Dacă **nu** există majoritar, e posibil ca toți să se anuleze
- La final, candidatul rămas este un **candidat posibil** — nu neapărat câștigător, trebuie verificat!

Traseu pas cu pas — $v = [7, 2, 7, 7, 4]$

i	$v[i]$	candidat	voturi
—	—	—	0
0	7	7	1
1	2	7	0
2	7	7	1
3	7	7	2
4	4	7	1

Reguli: $\text{voturi} = 0 \Rightarrow$ candidat nou; $v[i] = \text{candidat} \Rightarrow \text{voturi}++$; altfel $\Rightarrow \text{voturi}--$.

Candidat final: **7**. Urmează pasul de verificare.

Ideea de anulare

Imaginăm că împerechem valori diferite și le ștergem pe rând:

$v = [7, 2, 7, 7, 4]$

- Ștergem 7 cu 2: rămâne $[7, 7, 4]$
- Ștergem 7 cu 4: rămâne $[7]$

Dacă o valoare apare de mai mult de jumătate din numărul total de ori, ea **nu poate dispărea complet** — are prea mulți reprezentanți.

Algoritmul lui **Boyer-Moore** implementează exact această idee eficient, fără să memoreze efectiv perechile.

Algoritmul Boyer-Moore

```
1  int candidat = 0;
2  int voturi = 0;
3
4  for (int i = 0; i < n; i++) {
5      int x = v[i];
6      if (voturi == 0) {
7          candidat = x;
8          voturi = 1;
9      } else if (x == candidat) {
10         voturi++;
11     } else {
12         voturi--;
13     }
14 }
```

voturi **nu** e frecvența reală, ci balanța rămasă după anulări. Codul de mai sus ne dă doar un **candidat**.

Pasul de verificare

De ce e necesară verificarea? Pentru $v = [1, 2, 3, 4, 5]$, Boyer-Moore tot termină cu un candidat — dar nicio valoare nu apare de mai mult de jumătate din numărul de ori.

```
1  int frecventa = 0;
2  for (int i = 0; i < n; i++) {
3      if (v[i] == candidat) {
4          frecventa++;
5      }
6  }
7
8  if (frecventa > n / 2) {
9      cout << candidat;
10 } else {
11     cout << -1;
12 }
```

Cel mai frecvent bug: uitarea pasului de verificare. Condiția este strict mai mare decât $n/2$.

Complexitate — Element majoritar

Etapă	Timp	Spațiu
Găsire candidat (Boyer-Moore)	$O(n)$	$O(1)$
Verificare candidat	$O(n)$	$O(1)$
Total	$O(n)$	$O(1)$

Mult mai rapid decât abordarea naivă (numărare pentru fiecare valoare distinctă, $O(n^2)$ în cel mai rău caz).

Subsecvența de sumă maximă

Cerință

Dat un șir de n numere întregi (pot fi și negative), determinați suma maximă a unei **subsecvențe contigue** (elemente consecutive).

Exemplu: $v = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$

Subsecvența $[4, -1, 2, 1]$ are suma 6 — aceasta este suma maximă posibilă.

Soluție naivă

```
1 long long maxim = v[0];
2 for (int i = 0; i < n; i++) {
3     long long suma = 0;
4     for (int j = i; j < n; j++) {
5         suma += v[j];
6         maxim = max(maxim, suma);
7     }
8 }
9 cout << maxim;
```

- Încercăm **toate** subsecvențele posibile, calculând suma fiecăreia
- Complexitate: $O(n^2)$ — prea lent pentru n mare

Algoritmul lui Kadane

Pentru fiecare poziție, întrebarea e: continuăm subsecvența anterioară sau începem una nouă de aici?

```
1 long long sumaCurenta = v[0];
2 long long maxim = v[0];
3
4 for (int i = 1; i < n; i++) {
5     sumaCurenta = max((long long)v[i], sumaCurenta + v[i]);
6     maxim = max(maxim, sumaCurenta);
7 }
8 cout << maxim;
```

Dacă suma acumulată devine **mai mică** decât elementul curent, nu mai are sens s-o continuăm — pornim o subsecvență nouă.

Complexitate — Subsecvența de sumă maximă

Variantă	Timp	Spațiu
Naivă (toate subsecvențele)	$O(n^2)$	$O(1)$
Kadane	$O(n)$	$O(1)$

Recapitulare

Recapitulare

Temă	Timp	Spațiu
Fibonacci (iterativ)	$O(t)$	$O(1)$
Element majoritar (Boyer-Moore)	$O(n)$	$O(1)$
Subsecvența de sumă maximă (Kadane)	$O(n)$	$O(1)$

- Fibonacci recursiv recalculează aceleași subprobleme — varianta iterativă rezolvă asta în $O(1)$ spațiu
- La elementul majoritar, un candidat găsit prin anulare **trebuie verificat**
- La Kadane, decizia la fiecare pas: continuăm sau pornim din nou

Resurse

Șirul lui Fibonacci

- **pbinfo.ro** — Șirul lui Fibonacci (RO)
- **infoas.ro** — Șirul lui Fibonacci în C++ (RO)
- **cp-algorithms.com** — Fibonacci Numbers (EN, avansat)
- **wikipedia.org** — Fibonacci sequence (EN)

Elementul majoritar

- **infoarena.ro** — Problema majorității votului (RO)
- **tutoriale-pe.net** — Elementul majoritar (RO)
- **geeksforgeeks.org** — Boyer–Moore Majority Voting (EN)
- **wikipedia.org** — Boyer–Moore majority vote algorithm (EN)

Subsecvența de sumă maximă

- **pbinfo.ro** — Algoritmul lui Kadane (RO)
- **wikipedia.org** — Maximum subarray problem (EN)

Probleme de antrenament

Ușoare

- **Fibonacci #255 #759** (AlgoCode)
- **Majority Element #4219** (AlgoCode)

Medie

- **Kadane Segment Report #4362** (AlgoCode)

Olimpiadă

- **avarcolaci** (ONI 2014, clasa XI–XII)